Cognitive Load — API & Architecture Explained

A complete walkthrough of what this project serves, how the pieces fit together, and what every API endpoint does. Written for the `binary-class` branch.

---

1. What the system is

Three layers:

```
serving/  serving_3class/     the model bundles — weights + fitted scalers +
        (data on disk)        precomputed evaluation artifacts
              │
              ▼
backend/  (FastAPI, :8000)    loads each bundle once at startup, runs inference,
                              serves the artifacts as JSON
              │  HTTP
              ▼
frontend/ (React + Vite,      seven pages that read everything live from the API
           :5173)
```

The guiding rule of the project: **every number the UI shows traces to a committed artifact.** The frontend hardcodes no metrics.

Running it

```
# Terminal 1 — API
cd backend && ../.venv/bin/uvicorn main:app --reload --port 8000

# Terminal 2 — UI
cd frontend && npm run dev
```

Interactive API docs (auto-generated by FastAPI): http://localhost:8000/docs. That page is live and lets you execute any endpoint in the browser.

---

2. The two model bundles

The app serves **two different trained models**, selectable in the UI:

	`binary`	`3class`
Directory	`serving/`	`serving_3class/`
Classes	Low, High	Low, Medium, High
Task	0-back vs 3-back	0-back vs 2-back vs 3-back
Chance	50%	33%

Both consume the **same 1476 input features** — 540 EEG + 936 fNIRS (HbO), byte-identical column lists. Only the output head differs. This is why one fitted scaler is valid for both: scaling is label-agnostic.

What a bundle contains

File	Purpose
`model_config.json`	Architecture hyperparameters + class names. The server rebuilds the network from this.
`model_weights.pt`	The trained `state_dict`.
`feature_scaler.pkl`	Fitted `RobustScaler` (EEG) + `StandardScaler` (fNIRS), used for CSV uploads.
`label_map.json`	Class index → name.
`demo_samples.json`	Real held-out test windows, fully preprocessed — powers Live Prediction and Session Trend. **Git LFS.**
`feature_schema.json`	The exact column names an uploaded CSV must contain.
`upload_template.csv`	A downloadable example CSV in the right shape.
`frontend_data/*.json`	Precomputed evaluation artifacts: metrics, confusion matrices, SHAP, attention.

`serving/` additionally has `rf_model.pkl` (Git LFS, 46 MB) and `frontend_data/{diagnostics,research}.json`. The Random Forest is **not used** — `model_service.py` sets `self.rf = None`, so the served model is the Hybrid network alone despite `model_config.json` still describing an ensemble.

Git LFS

`serving/rf_model.pkl`, `serving/demo_samples.json` and `serving_3class/demo_samples.json` are stored in Git LFS. A clone without LFS gets 133-byte pointer files and the backend fails to start. Fix:

```
brew install git-lfs
git lfs install --local
git lfs pull
```

Provenance of the 3-class model — read this

The original 3-class research checkpoint (`outputs/hybrid_best.pt`) is **not in this repository**, so the bundle's current weights are a stand-in.

> **Update (August 2026).** The claim that it "cannot be regenerated because `data/` is absent" no longer holds: the extracted feature data **is** present under `ZDATASET/` and is referenced from `data/` by symlink. A genuine 3-class model can now be retrained from the real features. The stand-in described below is still what the bundle currently ships.

The 3-class weights currently in the bundle were produced by `train_3class_from_demo.py`, which trains the architecture described by `serving_3class/model_config.json` on the 7650 labelled windows inside `demo_samples.json` (6 subjects), validating on subjects never trained on.

It is a stand-in, not the research model:

- held-out-subject accuracy **41.3%**, macro F1 **0.388** (chance 33.3%)
- trained on what were originally the *fixed-split test* windows, since that is the only labelled 3-class data present
- 6 subjects, not 26

This is recorded in `model_config.json["provenance"]`, exposed by `GET /api/bundles`, and rendered as a banner on Overview, Live Prediction and Model Insights. The charts in the 3-class bundle still describe the **original** model, so they must not be read as this one's results.

---

3. API reference

Base URL `http://127.0.0.1:8000`. Every model-scoped endpoint accepts `?version=binary` (default) or `?version=3class`.

Error codes

Code	Meaning
400	Unknown `version` value.
404	The bundle has no such artifact (e.g. 3-class has no `diagnostics.json` or `research.json`).
422	Bad request body, or an uploaded CSV missing required columns.
503	The requested bundle exists but could not be loaded — response names the missing file.

Bundles load independently at startup. One broken bundle does not take down the API; it is marked unavailable and only its own requests return 503.

`GET /api/health`

Liveness check. `{"status": "ok"}`.

`GET /api/bundles`

Which models can actually be served. The UI's model switch reads this to decide which options to enable, and disables the rest **with the reason** rather than hiding them.


```
{
  "bundles": [
    {"version": "binary", "available": true,
     "num_classes": 2, "class_names": ["Low", "High"],
     "reason": null, "provenance": null},
    {"version": "3class", "available": true,
     "num_classes": 3, "class_names": ["Low", "Medium", "High"],
     "reason": null,
     "provenance": {"kind": "retrained_stand_in", "heldout_subject_accuracy": 0.4133, "...": "..."}}
  ],
  "default": "binary"
}
```


`class_names` is read from `model_config.json` even when a bundle is unavailable, so the switch can still name the option it greys out.

`GET /api/model/metrics`

The full evaluation record from `frontend_data/metrics.json`:

- `fixed_split` — accuracy / macro F1 / per-class recall / AUC on a fixed train-test split.
- `loso` — the honest **leave-one-subject-out** numbers, keyed by configuration (`hybrid`, `fnirs_only`, `hybrid_non_normalized`, `ensemble_hybrid_rf`), each with `f1_mean`, `f1_std`, `acc_mean`, `auc_mean`, `recall_mean`.
- `significance` — paired tests between configurations.

The gap between `fixed_split` and `loso` is the project's central finding: a fixed split leaks subject identity and flatters the model.

`GET /api/model/confusion-matrix?model=hybrid|eeg|fnirs`

One confusion matrix. Rows = true class, columns = predicted.

```
{"model": "hybrid", "labels": ["Low", "Medium", "High"], "matrix": [[27,19,2],[11,28,15],[20,19,9]]}
```

The response carries its **own** `labels`, and the UI renders those rather than the switch's class names — see the caveat in §7.

`GET /api/model/config`

The raw `model_config.json` for the bundle: layer dimensions, dropout, fusion type, class names, and `provenance` when present.

`GET /api/model/diagnostics`

Extra calibration / reliability artifacts. **404 on the 3-class bundle** — it has no `diagnostics.json`. The frontend treats this as optional and catches it.

`GET /api/explainability/shap`

SHAP feature-category importances — which feature families drive predictions.

`GET /api/explainability/attention`

Learned attention-gate values, i.e. how much the fusion layer weights EEG versus fNIRS.

`GET /api/research/summary`

The consolidated research dossier (DANN ablation, HbR, protocol benchmark, classical sweep) backing the Research page. Only packaged in `serving/`; the frontend falls back to the binary copy when 3-class is selected, since this is a project-level document rather than a per-model artifact.

`GET /api/demo/subjects`

Subjects available for playback, with window counts and ground-truth labels:

```
[{"subject_id": 1, "trial_id": 0, "num_windows": 204, "true_labels": [1,1,0,...],
  "true_label_names": ["High","High","Low", "..."]}]
```

`GET /api/demo/session/{subject_id}/{trial_id}`

The raw preprocessed windows for one session. 404 if the subject has none.

`POST /api/predict`

Single-window prediction. Supply **either** a demo pointer **or** raw features:

```
{"subject_id": 1, "window_index": 0}
{"features": [0.12, -0.4, "... 1476 floats ..."]}
```

Response:

```
{
  "predicted_label": "High",
  "predicted_index": 1,
  "probabilities": {"Low": 0.418, "High": 0.582},
  "attention_weights": {"eeg": 0.412, "fnirs": 0.546,
                        "eeg_share": 0.430, "fnirs_share": 0.570},
  "uncertainty": {"confidence": 0.582, "margin": 0.164, "entropy": 0.98,
                  "abstain": false, "abstain_confidence_threshold": 0.45,
                  "abstain_margin_threshold": 0.10},
  "true_label": 1, "true_label_name": "High", "window_index": 0
}
```


`uncertainty.abstain` is true when the model is not confident enough to commit — `confidence` below 0.45 or a top-2 `margin` under 0.10. It is reported rather than hidden, so the UI can say "not sure" instead of guessing.

Neither `features` nor a complete demo pointer → **422**.

`POST /api/predict/session`

Same, batched over an entire session: `{"subject_id": 1, "trial_id": 0}` returns one row per window with `timestamp_s`, prediction, probabilities, attention and ground truth. This drives the Session Trend chart.

`GET /api/predict/upload/template`

Downloads `upload_template.csv` — a correctly shaped example for the selected bundle.

`POST /api/predict/upload`

Batch prediction from a CSV of **already-extracted TU Berlin features** (`multipart/form-data`, field `file`).

Expected columns: the 540 EEG + 936 HbO feature columns, plus `subject_id` (for per-subject normalization) and optionally `Cognitive_Load` as ground truth. `hbr_*` and `task` columns are accepted and ignored.

Validation is strict. A CSV missing any required column is rejected with 422 and the exact list of what is missing — never silently processed. Without a `subject_id` column the response is flagged `"normalization": "global_fallback_no_subject_id"`, a documented degraded mode.

Response carries `summary.class_distribution`, `summary.accuracy_vs_uploaded_labels` (when labels were supplied) and a per-row `predictions` array.

Scope: extracted features only, TU-Berlin format. Not raw signals, not another dataset or device.

`POST /api/predict/upload_mat`

Takes raw `.mat` files (`eeg_file` + `nirs_file`), runs feature extraction via `mat_extractor.py`, then predicts. Slower — the client allows 5 minutes.

---

4. Frontend pages

Page	Route	Reads
Overview	`/`	`model/metrics` for both bundles
Research	`/research`	`research/summary`, `model/metrics`, `shap`, `attention`
Live	`/live`	`demo/subjects`, `predict`
Session	`/session`	`demo/subjects`, `predict/session`
Upload	`/upload`	`predict/upload`, `predict/upload_mat`, template
Insights	`/insights`	`confusion-matrix` ×3, `shap`, `attention`, `metrics`, `diagnostics`
Methodology	`/methodology`	`metrics`, `diagnostics`

The model switch

`lib/modelVersion.jsx` holds the selected version in React context, persisted to `localStorage` under `cl-model-version`. `components/ModelSwitch.jsx` renders it in two variants — `hero` on the landing page, `compact` in the nav bar.

The choice is **resolved against** `/api/bundles`: if a stored pick refers to a bundle that has since gone missing, it falls back to an available one rather than leaving every page erroring. Class names, the chance baseline (50% vs 33%), the hero copy and the footer all follow the active bundle.

Live Prediction shows the selected model as primary and the *other* bundle alongside when it is also servable — so it works as a single-model demo or as a 2-class vs 3-class comparison, in either direction.

---

5. Evaluation runs — the experiment archive

Added with the `research/` framework (see `BINARY_MODEL_IMPROVEMENT.md`). These endpoints serve artifacts a training run wrote to disk under `outputs/runs/<run_id>/`. Nothing is computed in the request path, so the API can never disagree with the artifacts, and every metric traces back to that run's `predictions.csv`.

A run is **self-describing**: its `dataset_info.json` carries the class names, the classification mode and the evaluation protocol. So binary and 3-class runs go through the same code path with no per-model branching, and the frontend reads labels from the run rather than assuming a class count.

`GET /api/runs`

Run history, newest first. Optional `mode` (e.g. `binary`) and `status` filters.

```
{
  "runs": [{
    "run_id": "run_20260819_175820_33bd",
    "status": "completed", "stale": false, "stage": "done",
    "tag": "binary-improvement-full",
    "created_at": "...", "dataset": "tuberlin_sliding.csv",
    "classification_mode": "binary", "class_names": ["Low", "High"],
    "n_subjects": 26, "n_samples": 4576, "n_experiments": 35,
    "protocol": "26-fold LOSO over all subjects",
    "warnings": [],
    "metrics": {"loso_f1_macro": 0.0, "lockbox_f1_macro": 0.0, "...": "..."}
  }],
  "total": 1, "runs_root": "/.../outputs/runs", "modes": ["binary"]
}
```


`stale` is computed, not stored: a run that recorded itself as `running` but has not written an update in 30 minutes had its process killed. The status on disk is left untouched.

The rest

Endpoint	Returns
`GET /api/runs/{id}`	metadata, config, dataset info, leakage audit, and a manifest of what the run actually produced
`GET /api/runs/{id}/metrics`	pooled held-out metrics + plain-English explanations + LOSO summary + per-class
`GET /api/runs/{id}/confusion-matrix`	counts, row-normalised counts, labels, protocol
`GET /api/runs/{id}/curves`	ROC and precision-recall points (thinned for the browser)
`GET /api/runs/{id}/training-history`	per-fold curves and the fold-averaged curve
`GET /api/runs/{id}/per-subject`	per-subject held-out metrics, fold aggregate, lockbox
`GET /api/runs/{id}/feature-importance`	SHAP ranking, families, channels, modality share
`GET /api/runs/{id}/representations`	t-SNE coordinates per variant + subject-invariance probe
`GET /api/runs/{id}/experiments`	the leaderboard and the stage log (what was accepted, what was rejected)
`GET /api/runs/{id}/experiments/{exp_id}`	one arm's full record
`GET /api/runs/{id}/predictions`	raw saved predictions, paged (`limit` , `offset` , `subject`)
`GET /api/runs/{id}/summary`	`run_summary.json`
`GET /api/runs/{id}/report`	`run_summary.md` as text
`GET /api/runs/{id}/figure?path=...`	a PNG the run rendered; path-guarded to that run's directory, `.png` only

Error codes. 404 for an unknown run, or for an artifact the run never produced (the message names the expected file and notes the run may have failed before that stage). 400 for a non-PNG figure request or an escaping path.

Metric validity is data-driven

`metrics.json` carries an `unavailable` map explaining anything it could not compute, rather than emitting a fabricated number:

```
"unavailable": {
  "roc_auc_macro_ovo": "one-vs-one is undefined for binary classification (identical to OVR)",
  "top_3_accuracy": "Not applicable: K=3 is not smaller than the number of classes (2)."
}
```

The Evaluation page renders those strings in place of a value.

---

6. Frontend pages added

Page	Route	Reads
Evaluation Metrics	`/evaluation`	every `runs/{id}/...` endpoint for the selected run
Run History	`/runs`	`runs`

The Evaluation page's **run selector** controls the entire page: selecting a run reloads every section from that run alone, so run A's metrics can never appear next to run B's plots. The selected run id is mirrored into the URL (`/evaluation?run=<id>`) so a view is linkable, and Run History rows link straight into it.

Class names, class count, chance level and which metrics are valid all come from the selected run's artifacts — the page has no `["Low","Medium","High"]` constant and no `if binary` branch.

---

7. Known caveats

The binary bundle's evaluation artifacts are partly stale 3-class data.

- `serving/frontend_data/confusion_matrices.json` is *entirely* 3-class: `labels: ["Low","Medium","High"]` , all matrices 3×3, even under `?version=binary` .
- `serving/frontend_data/metrics.json` is mixed: `loso.hybrid` and `loso.ensemble_hybrid_rf` are genuinely binary (2 recall values), while `fixed_split.*` , `loso.fnirs_only` and `loso.hybrid_non_normalized` are stale 3-class (3 values).

The UI contains this rather than papering over it: confusion charts render each matrix's own `labels` , so a stale matrix is labelled truthfully as 3-class; and the per-class recall chart drops any configuration whose recall array length disagrees with the class count instead of mislabelling the bars. **The real fix is regenerating these artifacts from a binary run of `export_for_serving.py` , which needs `data/` .**

Per-subject normalization is a demo-mode simplification. The model normalizes each subject against that subject's own statistics. A genuinely new person has none, so the demo reuses each known subject's precomputed stats.

Only 26 subjects in the original study — small for cross-subject generalization claims. The 3-class stand-in sees only 6.

The Random Forest half of the "ensemble" is disabled in code, though `model_config.json` and some API responses still say `hybrid_rf_ensemble` .

---

8. File map

```
backend/
  main.py            FastAPI app, all routes, per-bundle startup loading
  runs_service.py    read-only access to the outputs/runs/ archive
  model_service.py   rebuilds the network from config, runs inference
  data_service.py    reads frontend_data/*.json and demo_samples.json
  upload_service.py  CSV validation + preprocessing (mirrors data_loader.py)
  mat_extractor.py   raw .mat → features
  schemas.py         Pydantic request/response models

frontend/src/
  App.jsx                       nav, routes, footer
  lib/modelVersion.jsx          model-selection context
  components/ModelSwitch.jsx    the 2-class ↔ 3-class control
  components/ModelProvenanceNote.jsx  stand-in-model banner
  components/evaluation/        run selector, metric cards, evaluation charts
  api/client.js                 every API call
  pages/                        nine pages (incl. EvaluationMetrics, RunHistory)

research/                       leakage-audited experiment framework
run_campaign.py                 the binary-model improvement campaign
train_3class_from_demo.py       rebuilds the 3-class bundle (see §2)
export_for_serving.py           rebuilds serving/ — needs data/
export_3class.py                rebuilds serving_3class/ — needs data/
```